When Do More Threads Help? OpenMP Parallel Matrix Multiplication on a Hybrid P-core/E-core CPU, with an AVX2 Vectorisation Extension

Big Data – Individual Assignment 3: Parallel Matrix Multiplication and Optional Vectorisation

Aimar Daniel Alejandro Santana
Grado en Ciencia e Ingeniería de Datos, Universidad de Las Palmas de Gran Canaria
Repository: https://github.com/aimarsantana/BigData-Assignment03-MatMul
November 2026

Abstract. We study how the number of threads affects a shared-memory dense binary64 matrix multiplication in C on one machine with a hybrid CPU (Intel Core i7-13700K: 8 performance cores with 2 hardware threads each and 8 efficiency cores, 24 logical processors). The sequential baseline is the fastest kernel of Assignment 2 (ikj order, k loop unrolled by 4); the parallel version runs the same row kernel and distributes the rows of C over OpenMP threads, pinned P-cores first, then E-cores, then the second hardware thread of each P-core. All 290 configurations (n = 64–4096, 1–48 threads, two schedules, three placements, scalar and AVX2 builds) give bit-identical results. With one thread the OpenMP version costs at most 6 % more than the sequential one. On the 8 P-cores the speed-up is nearly linear for 256 ≤ n ≤ 1536 (S(8) = 6.7–7.8), lower once B no longer fits in the 30 MB L3 cache (4.2–4.8 for n ≥ 2048), and negligible for n = 64. Adding E-cores with a static schedule *reduces* the speed-up (n = 2048: 4.75 with 8 threads, 3.02 with 12) because every thread gets the same number of rows and an E-core is 2.3× slower; a dynamic schedule removes this drop and reaches 12.2× with 24 threads (static: 6.0×). AVX2 vectorisation, separated from the thread effect, is worth 1.9–3.1× when the data fit in the private caches (n ≤ 512) but only 1.1–1.2× for n ≥ 1024. Memory use does not depend on the thread count (+ ≈30 KiB per thread). Conclusions are limited to this kernel, compiler and machine.

1. Introduction and objective

Assignment 1 compared a basic triple loop in three languages; Assignment 2 showed that a sequential C kernel becomes 16–24× faster through loop reordering and unrolling, and that it then becomes limited by memory access rather than arithmetic. A modern processor offers two further sources of speed: several cores (thread-level parallelism) and SIMD units inside each core (data-level parallelism). This study measures what the first one buys for the Assignment 2 kernel, on a desktop processor whose cores are of two different types, and separates it from the second one, which we evaluate as the optional extension. Matrices are n×n; n is the side length; p is the number of threads.

Research question. *How does the thread count p affect the runtime, speed-up, efficiency and memory of an OpenMP parallel dense binary64 multiplication on a hybrid 8 P-core + 8 E-core CPU, for which matrix sizes does parallel execution help, and how much does AVX2 vectorisation add on top of it?*

Predictions (formulated before the main runs; a single short probe at n = 2048 with 1, 8, 16 and 24 threads, used to choose the experiment list, had already shown that 16 threads were slower than 8, which motivated H3):

1. **H1.** On P-cores only (p ≤ 8) the speed-up is close to p (E(p) ≥ 0.8) for medium sizes, because the rows of C are independent and the parallel overhead is O(p) against O(n³) work.
2. **H2.** For small matrices (n ≤ 128) the cost of starting and synchronising the threads is comparable to the work, so the speed-up stays low and falls with more threads.
3. **H3.** With a static schedule, adding the slower E-cores (8 < p ≤ 16) lowers the speed-up, because the slowest thread determines the time; a dynamic schedule recovers it.
4. **H4.** When B no longer fits in the last-level cache (n ≥ 2048), the efficiency falls even on P-cores, because the threads share the cache and memory bandwidth.

5. **H5.** AVX2 (4 doubles per instruction) gives a large gain (2–4×) only while the data are cache resident, and the gain shrinks when the kernel is bandwidth limited, including when many threads run.

2. Background

Shared-memory parallelism. In C=AB every row of C depends only on one row of A and the whole of B, so the rows can be computed concurrently without communication; only the read-only operands are shared. OpenMP [1, 2] expresses this with `#pragma omp parallel for`: a team of p threads is created (and kept in a pool), and the iterations are divided by a *schedule*. `static` gives each thread one contiguous block of about n/p iterations; `dynamic,1` hands out one iteration at a time to whichever thread is free, adding a small synchronisation cost per iteration but balancing unequal speeds. With T_{seq} the sequential time and T_p the time with p threads, the speed-up is $S(p) = T_{seq}/T_p$ and the efficiency $E(p) = S(p)/p$; Amdahl's law [3] bounds S(p) by the serial fraction of the work, which here is tiny (thread start-up, zeroing is parallel too).

Hardware limits. Parallel threads share the last-level cache and the memory bandwidth. The roofline model [4] bounds the attainable throughput by min(peak compute, bandwidth × arithmetic intensity). The ikj/unroll4 kernel reads all of B ($8n^2$ bytes) for every row of C, i.e. $8n^3$ bytes per product for $2n^3$ flops (0.25 flop/byte), so it is bandwidth-bound unless B is reused from cache [5, 6]. **SMT** (Hyper-Threading) lets two threads share one core's execution units, which helps mainly when a thread stalls on memory [5]. **Hybrid CPUs** such as Intel's 13th generation combine large performance cores with smaller efficiency cores; Windows reports them through the `EfficiencyClass` of each core [7, 8]. Equal work per thread then means unequal finishing times. **SIMD vectorisation** applies one instruction to several data: an AVX2 `vmulpd` on a 256-bit ymm register processes 4 doubles, against 1 for scalar SSE2 `mulsd` [8]. A loop over j in C[i][j] += a·B[k][j] vectorises without changing the order of the additions of any C[i][j], provided no fused multiply-add (FMA) is generated.

3. Methodology

3.1 Implementations

Table 1. Methods (all in `src/c/matmul3.c`, C11, binary64, own code; no numerical library).

Method	Description
seq	**Sequential baseline**: the `unroll4` kernel of Assignment 2 (ikj order, k loop unrolled by 4, row-pointer storage `double**`), moved unchanged into a function `row_unroll4` that computes one row of C (including zeroing it).
par	**Parallel version**: `#pragma omp parallel for schedule(runtime)` over the rows, each iteration calls the same `row_unroll4`. Schedule `static` (default) or `dynamic,1`. Threads write disjoint rows of C; A and B are read-only; there is no lock, atomic or reduction in the kernel.
ijk	Basic triple loop of Assignment 1 (context only, run up to n = 2048; it is not the baseline of S(p)).

Both seq and par use the same algorithm, numeric type, storage layout, input files and timing boundary; the only difference is the distribution of rows over threads. The OpenMP team is created, sized (`omp_set_dynamic(0)`, `omp_set_num_threads(p)`) and pinned before the first warm-up, outside the timer, and the harness removes all `OMP_*` environment variables, so the actual thread count is the requested one (checked in every run: 0 mismatches). No library creates internal threads in any timed region.

Two builds (vectorisation extension). The same source is compiled twice with gcc 16.1.0: *scalar* (`-O2 -fopenmp -fno-tree-vectorize`) and *AVX2* (`-O2 -fopenmp -mavx2 -ffp-contract=off`, plus `#pragma omp simd` on the two inner j loops). Evidence that vectorisation occurs only in the AVX2 build: (i) the gcc vectoriser report (`-fopt-info-vec-optimized`) lists the j loops as "vectorized using 32 byte vectors" for AVX2 and nothing for the scalar build; (ii) in the disassembly (`objdump -d`, `results/disasm_summary.csv`) the functions `matmul_seq` and the OpenMP body `matmul_par._omp_fn.0` contain 10 scalar `mulsd/addsd` and no packed instruction in the scalar build, and 10 `vmulpd/vaddpd` on ymm registers (main loop), 10 on xmm (epilogue) and 10 scalar ones (remainder) in the AVX2 build; (iii) neither build contains an FMA instruction (0), so the

arithmetic, and the results, are identical. The two builds are compared at equal p, schedule and placement, so the SIMD effect is separated from the thread effect.

3.2 Inputs

The dense matrices of Assignments 1 and 2: NumPy PCG64 (`default_rng`), seeds 20261012 (A) and 20261013 (B), U[−1,1), binary64, little-endian row-major files, n ∈ {64, 128, 256, 512, 1024, 1536, 2048, 3072, 4096}. The SHA-256 of every regenerated file equals the manifest of Assignment 2, and that of Assignment 1 for the sizes it contains (`data/inputs/manifest.json`). Every method reads the same files.

3.3 Environment and thread placement

Table 2. Measurement environment (`results/environment.txt`, `results/topology.csv`).

CPU	Intel Core i7-13700K (desktop): 8 P-cores × 2 hardware threads + 8 E-cores × 1 = 16 physical cores, 24 logical processors; L2 2 MB per P-core and 4 MB per cluster of 4 E-cores (24 MB total reported); L3 30 MB shared. Turbo and frequency not controlled.
Memory / OS	31.7 GB RAM (2×16 GB DDR5-5600, theoretical peak 89.6 GB/s); Windows 11 Home build 26200; "Balanced" power plan; process priority HIGH.
Toolchain	gcc 16.1.0 (WinLibs MinGW-w64 UCRT, POSIX threads) with libgomp (`_OPENMP` = 202111); Python 3.11.9, NumPy 2.4.6, pandas 3.1.1, matplotlib 3.11.2 (inputs, validation, analysis only).
Placement	Logical processors 0–15 are the 8 P-cores (pairs 0/1, 2/3, …), 16–23 the E-cores. Policy *pcores*: thread t → slot t mod 24 of the order 0,2,…,14 (P-cores) | 16–23 (E-cores) | 1,3,…,15 (second SMT threads); so p ≤ 8 uses P-cores only, p = 10–16 adds E-cores, p = 20–24 adds SMT, p = 32, 48 oversubscribes. Policy *ecores*: E-cores first. Policy *none*: no affinity. Each thread pins itself with `SetThreadAffinityMask`; at the end of each run every thread reports its processor: 256 of 256 pinned runs ran exactly where intended. The sequential baseline runs on the same logical processor as thread 0 (logical 0).

Experiments (`config/benchmark_config.json`): **P1** scalar build, all sizes, p ∈ {1,2,4,6,8,10,12,16,20,24,32,48}, static, pcores (+ seq, + ijk for n ≤ 2048); **V1** the same grid with the AVX2 build; **P2** as P1 with `dynamic,1` for n ∈ {1024, 2048, 4096}; **P3** n = 2048 without pinning; **P4** n = 2048 on E-cores first (p ≤ 8). For a given n all configurations of an experiment run back to back, so T_{seq} and T_p come from the same session.

3.4 Correctness and safe access to shared data

`tests/validate.py` ran 782 checks before benchmarking (**782 pass, 0 fail**): a hand-computed 3×3 product, identity (left and right) and zero matrices, sizes 1, 2, 3, 5, 7, 33, 100, 127, 130 (not multiples of the unroll factor or of p, and fewer rows than threads), 1–48 threads, both schedules, all placements, both builds, and the benchmark inputs at n = 512 and 1024. The reference is NumPy (OpenBLAS), used only for checking, with tolerance $|\Delta C| \le 10^{-9} + 10^{-9}|C|$, above the rounding bound $n^2u \approx 1.2\times10^{-10}$ for n = 1024 [9] (largest error on the benchmark inputs 1.4e-13). The stronger test is that every configuration must equal the scalar sequential result *bit for bit* (776 comparisons, all equal): a data race or a wrong row partition would break it. As a race check, the parallel kernel was run 20 times in one process with 16 (static), 24 and 48 threads (dynamic) in both builds, and all repetitions gave identical checksums. During the benchmarks, the checksum of all 2226 measured samples of the 290 configurations was compared per n: 0 disagreements.

3.5 Measurement protocol

- **Timer boundary (kernel only).** The multiplication including zeroing of C (row by row, inside the parallel loop for par). Input loading, allocation of C, creation and pinning of the thread team, and the result checksum are outside the timer. Timer: `QueryPerformanceCounter` (monotonic).
- **Repetitions.** One fresh process per configuration; warm-up 2 runs (n ≤ 512) or 1 run (excluded from statistics, kept in the raw data); 10 measured repetitions for n ≤ 512, 7 for n ≤ 2048, 5 above. Short kernels are batched automatically so that a sample lasts ≈20 ms; the time per multiplication is reported.
- **Statistics.** Median and interquartile range (IQR). **S(p) and E(p) are computed from median times**; T_{seq} is the seq run of the same build (P2 and P4 use the seq run of P1, measured in the same session just before). All 2226 measured samples are in `results/raw_samples.csv`; none was removed.

- **Budgets.** 120 s per sample and 8 GB peak working set, with a stop rule per series. All 290 of 290 configurations completed within the budget; the largest tested size is n = 4096 (also the largest attempted), chosen so that the whole campaign fits in about 45 minutes.
- **Memory metric.** Peak working set (`PeakWorkingSetSize`) of the process, read by the process itself through `GetProcessMemoryInfo` [10]; it includes the runtime, the loaded inputs, the thread stacks and all temporaries.

4. Results

The median relative IQR over all configurations is 1.8 % (95th percentile 11 %); the largest spreads (up to 45 %) occur in oversubscribed runs (p = 32, 48), where threads are time-sliced, and at n ≤ 128.

4.1 Runtime, speed-up and efficiency (P1, scalar build, static schedule)

Table 3. Scalar build (P1): median time per multiplication, IQR below in grey (10 repetitions for n ≤ 512, 7 for n ≤ 2048, 5 above). seq = sequential baseline; p = threads of the parallel version (static schedule, P-cores first). ijk = basic algorithm of Assignment 1, context only, run up to n = 2048.

n	ijk (A1)	seq	p=1	p=2	p=4	p=8	p=12	p=16	p=24	p=32	p=48
64	65.4 µs	36.9 µs 0.8 µs	39.4 µs 0.1 µs	35.9 µs 0.3 µs	31.2 µs 1.2 µs	35.3 µs 2.2 µs	78.2 µs 8.6 µs	103.6 µs 6.2 µs	135.5 µs 9.9 µs	141.0 µs 18.8 µs	164.8 µs 63.1 µs
128	932.4 µs	293.5 µs 5.6 µs	297.8 µs 1.4 µs	168.0 µs 1.7 µs	98.4 µs 1.5 µs	71.9 µs 2.0 µs	125.9 µs 9.2 µs	139.4 µs 12.8 µs	155.4 µs 3.4 µs	173.8 µs 11.9 µs	237.7 µs 14.1 µs
256	9.45 ms	2.34 ms 14.8 µs	2.35 ms 10.9 µs	1.23 ms 14.0 µs	633.4 µs 3.5 µs	351.0 µs 33.8 µs	520.8 µs 9.9 µs	445.4 µs 10.1 µs	380.6 µs 7.7 µs	376.1 µs 42.4 µs	436.1 µs 8.9 µs
512	103.59 ms	19.15 ms 129.0 µs	19.97 ms 81.1 µs	10.11 ms 52.2 µs	5.07 ms 127.0 µs	2.74 ms 66.9 µs	3.70 ms 251.4 µs	2.89 ms 82.5 µs	2.01 ms 57.1 µs	2.28 ms 219.1 µs	2.08 ms 39.9 µs
1024	2.37 s	166.90 ms 249.3 µs	167.46 ms 864.0 µs	82.11 ms 239.3 µs	42.35 ms 89.9 µs	21.47 ms 147.6 µs	30.78 ms 418.3 µs	23.52 ms 639.8 µs	16.23 ms 698.4 µs	15.72 ms 138.7 µs	16.61 ms 266.2 µs
1536	8.64 s	619.23 ms 8.35 ms	618.47 ms 6.50 ms	315.72 ms 13.03 ms	158.63 ms 2.59 ms	83.62 ms 2.60 ms	143.75 ms 5.19 ms	109.52 ms 6.00 ms	67.32 ms 2.57 ms	57.55 ms 12.93 ms	70.83 ms 8.47 ms
2048	72.43 s	1.87 s 4.96 ms	1.86 s 10.44 ms	1.15 s 23.82 ms	602.83 ms 9.00 ms	393.14 ms 14.47 ms	619.05 ms 22.39 ms	492.57 ms 4.07 ms	313.21 ms 5.35 ms	213.87 ms 10.54 ms	306.17 ms 8.35 ms
3072	–	6.83 s 4.56 ms	6.82 s 2.41 ms	4.41 s 28.36 ms	2.46 s 5.54 ms	1.62 s 6.09 ms	2.74 s 815.4 µs	2.05 s 13.39 ms	1.37 s 15.20 ms	1.00 s 16.91 ms	1.40 s 26.91 ms
4096	–	17.28 s 123.65 ms	17.24 s 6.70 ms	11.89 s 256.35 ms	6.12 s 27.27 ms	3.94 s 3.02 ms	7.01 s 6.83 ms	5.25 s 17.64 ms	3.48 s 38.20 ms	2.54 s 63.93 ms	3.64 s 60.60 ms

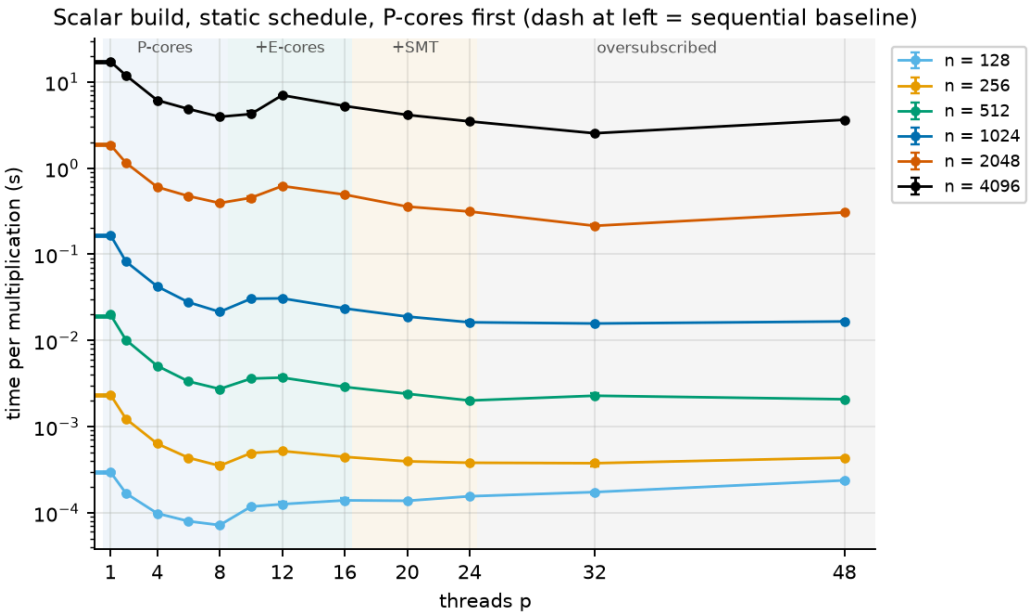


Figure 1. Time per multiplication versus threads (median, bars = IQR, mostly smaller than the markers). The short dash at the left of each curve is the sequential baseline. Shaded regions: P-cores only, P-cores + E-cores, + SMT threads, oversubscribed.

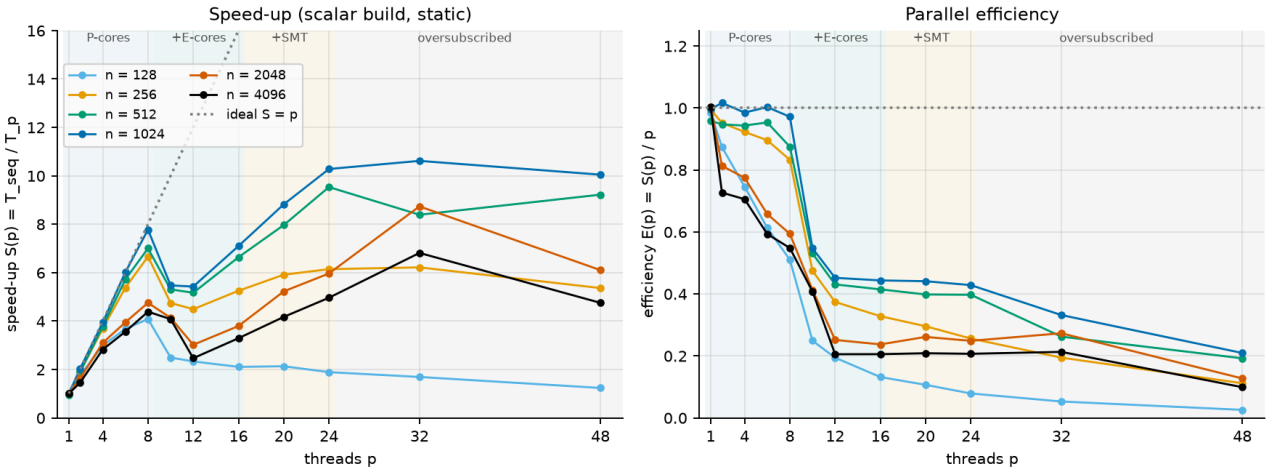


Figure 2. Speed-up S(p) and efficiency E(p) of the scalar build with the static schedule (from medians).

Table 4. Speed-up S(p) / efficiency E(p) of the scalar build (P1), from median times. p = 1 shows the OpenMP overhead (S(1) < 1 means the parallel version with one thread is slower than the sequential one).

n	p=1	p=2	p=4	p=8	p=12	p=16	p=24	p=32	p=48
64	0.94 / 0.94	1.03 / 0.51	1.18 / 0.30	1.05 / 0.13	0.47 / 0.04	0.36 / 0.02	0.27 / 0.01	0.26 / 0.01	0.22 / 0.00
128	0.99 / 0.99	1.75 / 0.87	2.98 / 0.75	4.08 / 0.51	2.33 / 0.19	2.11 / 0.13	1.89 / 0.08	1.69 / 0.05	1.23 / 0.03
256	1.00 / 1.00	1.90 / 0.95	3.69 / 0.92	6.66 / 0.83	4.49 / 0.37	5.25 / 0.33	6.14 / 0.26	6.22 / 0.19	5.36 / 0.11
512	0.96 / 0.96	1.89 / 0.95	3.77 / 0.94	7.00 / 0.87	5.17 / 0.43	6.63 / 0.41	9.53 / 0.40	8.39 / 0.26	9.22 / 0.19
1024	1.00 / 1.00	2.03 / 1.02	3.94 / 0.99	7.77 / 0.97	5.42 / 0.45	7.10 / 0.44	10.28 / 0.43	10.62 / 0.33	10.05 / 0.21
1536	1.00 / 1.00	1.96 / 0.98	3.90 / 0.98	7.41 / 0.93	4.31 / 0.36	5.65 / 0.35	9.20 / 0.38	10.76 / 0.34	8.74 / 0.18
2048	1.00 / 1.00	1.63 / 0.81	3.10 / 0.77	4.75 / 0.59	3.02 / 0.25	3.79 / 0.24	5.96 / 0.25	8.74 / 0.27	6.10 / 0.13
3072	1.00 / 1.00	1.55 / 0.77	2.78 / 0.70	4.21 / 0.53	2.50 / 0.21	3.33 / 0.21	5.00 / 0.21	6.81 / 0.21	4.88 / 0.10
4096	1.00 / 1.00	1.45 / 0.73	2.82 / 0.71	4.38 / 0.55	2.46 / 0.21	3.29 / 0.21	4.96 / 0.21	6.81 / 0.21	4.75 / 0.10

Overhead. With one thread the OpenMP version takes 0.94–1.00 times the sequential time (S(1) = 0.94 at n = 64, ≥ 0.96 from n = 128): the cost of the parallel loop with one thread is a few microseconds per product. **P-cores (p ≤ 8).** For 256 ≤ n ≤ 1536, S(8) = 6.66–7.77 and E(8) = 0.83–0.97; for n ≥ 2048 it falls to 4.75 (2048), 4.21 (3072) and 4.38 (4096), with an efficiency of at most 0.82 already at p = 2. **E-cores (p = 10–16).** At every size n ≥ 128 the speed-up *drops* when the first E-cores join (n = 1024: 7.77 → 5.47 at p = 10)

and recovers only partly by p = 16. **SMT (p = 20–24)** raises S again (10.28 at n = 1024). **Oversubscription:** p = 32 gives the best static result for n ≥ 1024 (e.g. 8.74 at n = 2048); p = 48 is worse again. For context, the basic ijk loop of Assignment 1 takes 72.4 s at n = 2048, 39× the sequential baseline.

4.2 When does parallel execution help? (matrix size)

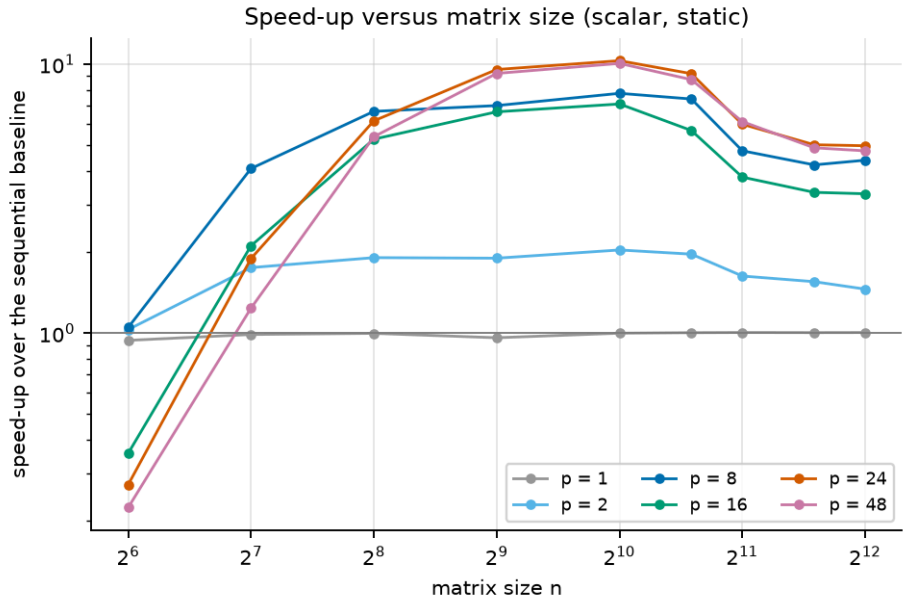


Figure 3. Speed-up versus n for selected thread counts (scalar, static). Below the horizontal line the parallel version is slower than the sequential one.

At n = 64 (37 µs sequential) the best speed-up is 1.18 (p = 4) and from p = 10 the parallel version is slower than the sequential one (S(24) = 0.27). At n = 128 (0.29 ms) 8 P-cores give 4.08, but more threads again lose (1.89 with 24). From n = 256 on, every p ≥ 2 is faster than the baseline. The largest speed-ups (10–11) are reached for n = 1024–1536, and they decrease for larger matrices.

4.3 Static versus dynamic scheduling (P2)

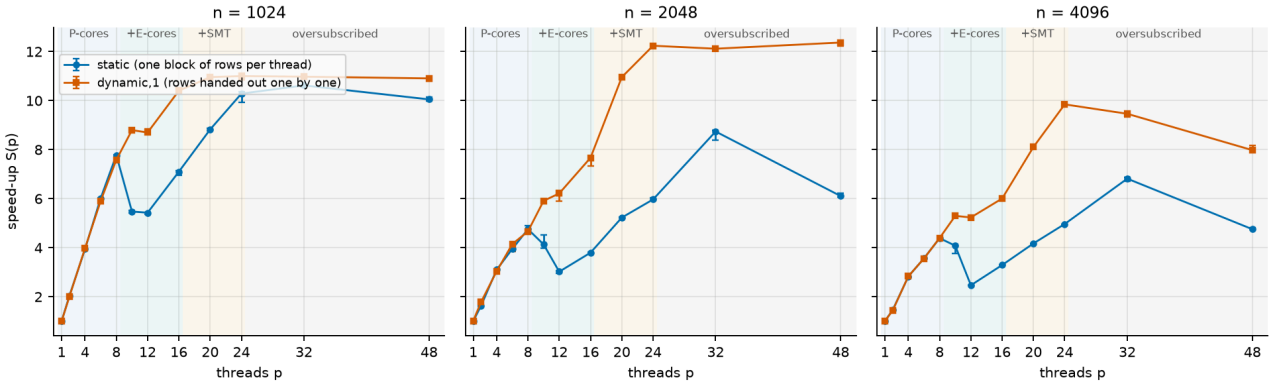


Figure 4. Speed-up with the static and the dynamic,1 schedule (P-cores first, scalar build); bars = speed-up computed from the quartiles of T_p.

Table 5. Static versus dynamic scheduling (scalar build, P-cores first; P1 versus P2).

n	p	S static	S dynamic,1	T static / T dynamic
1024	8	7.77	7.58	0.97×
1024	12	5.42	8.69	1.60×
1024	16	7.10	10.40	1.47×
1024	20	8.82	10.95	1.24×
1024	24	10.28	11.00	1.07×
1024	32	10.62	10.97	1.03×
1024	48	10.05	10.90	1.09×
2048	8	4.75	4.66	0.98×
2048	12	3.02	6.21	2.06×
2048	16	3.79	7.66	2.02×
2048	20	5.22	10.95	2.10×
2048	24	5.96	12.23	2.05×
2048	32	8.74	12.11	1.39×
2048	48	6.10	12.36	2.03×
4096	8	4.38	4.39	1.00×
4096	12	2.46	5.23	2.12×
4096	16	3.29	6.00	1.83×
4096	20	4.17	8.10	1.94×
4096	24	4.96	9.84	1.98×
4096	32	6.81	9.46	1.39×
4096	48	4.75	7.97	1.68×

Up to p = 8 the two schedules differ by at most 9 % (within 3 % at p = 8; n = 1024–4096). Once E-cores participate (p = 10–24), dynamic scheduling is 1.07–1.6× faster than static at n = 1024 and 1.3–2.1× at n ≥ 2048 (n = 2048, p = 12: S = 6.21 versus 3.02), and the speed-up then grows monotonically up to p = 24: 11.0 (n = 1024), 12.2 (2048) and 9.8 (4096). With dynamic scheduling oversubscription brings nothing (n = 4096: 9.5 at p = 32, 8.0 at 48).

4.4 Thread placement and core types (P3, P4; n = 2048)

Table 6. n = 2048, scalar build: median time and, in parentheses, speed-up over the sequential baseline on a P-core (P3 uses its own unpinned sequential run). P4 with p ≤ 8 runs on E-cores only.

p	P-cores first, static (P1)	P-cores first, dynamic (P2)	not pinned, static (P3)	E-cores first, static (P4)
1	1.86 s (1.00)	1.84 s (1.01)	1.86 s (1.00)	4.25 s (0.44)
2	1.15 s (1.63)	1.05 s (1.78)	1.05 s (1.78)	2.11 s (0.89)
4	602.83 ms (3.10)	615.05 ms (3.04)	586.52 ms (3.19)	1.95 s (0.96)
8	393.14 ms (4.75)	400.70 ms (4.66)	369.85 ms (5.05)	1.08 s (1.72)
12	619.05 ms (3.02)	300.82 ms (6.21)	287.89 ms (6.49)	–
16	492.57 ms (3.79)	243.74 ms (7.66)	241.95 ms (7.72)	–
24	313.21 ms (5.96)	152.70 ms (12.23)	205.99 ms (9.07)	–

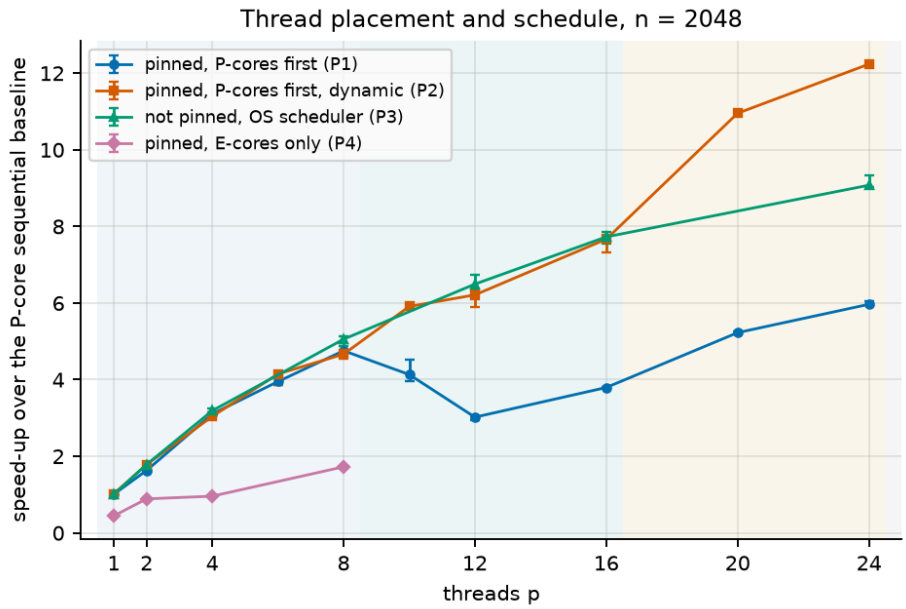


Figure 5. n = 2048: speed-up over the P-core sequential baseline for the placement and schedule variants.

The sequential kernel takes 1.87 s on a P-core and 4.22 s on an E-core, i.e. one E-core delivers 0.44 of a P-core. E-cores scale poorly inside one cluster: 2 E-cores give 0.89 and 4 E-cores (one cluster sharing a 4 MB L2) only 0.96; 8 E-cores (two clusters) give 1.72, so per E-core 0.24 and 0.22 P-core equivalents at p = 4 and 8. Without pinning (P3), the static schedule behaves like the dynamic one up to 16 threads (S(16) = 7.72 versus 7.66) and is better than the pinned static runs from p = 8, but it falls behind the pinned dynamic version at p = 24 (9.07 versus 12.23).

4.5 Vectorisation extension (V1 versus P1)

Table 7. Vectorisation extension (V1 versus P1). The SIMD gain compares the two builds at one thread; ‘S thread-only’ is the speed-up of the best AVX2 parallel run over the AVX2 sequential run, so the two effects are reported separately; the last column is their combination.

n	seq scalar	seq AVX2	SIMD gain (1 thread)	best scalar parallel	best AVX2 parallel	S thread-only (AVX2)	combined vs seq scalar
64	36.9 µs	13.2 µs	2.80×	31.2 µs (p=4)	13.1 µs (p=1)	1.0×	2.8×
128	293.5 µs	156.9 µs	1.87×	71.9 µs (p=8)	48.9 µs (p=4)	3.2×	6.0×
256	2.34 ms	762.2 µs	3.07×	351.0 µs (p=8)	160.5 µs (p=8)	4.7×	14.6×
512	19.15 ms	8.50 ms	2.25×	2.01 ms (p=24)	1.01 ms (p=8)	8.4×	18.9×
1024	166.90 ms	136.26 ms	1.22×	15.72 ms (p=32)	11.15 ms (p=32)	12.2×	15.0×
1536	619.23 ms	513.04 ms	1.21×	57.55 ms (p=32)	44.18 ms (p=32)	11.6×	14.0×
2048	1.87 s	1.69 s	1.11×	213.87 ms (p=32)	148.40 ms (p=32)	11.4×	12.6×
3072	6.83 s	6.11 s	1.12×	1.00 s (p=32)	768.94 ms (p=32)	7.9×	8.9×
4096	17.28 s	14.93 s	1.16×	2.54 s (p=32)	2.17 s (p=24)	6.9×	8.0×

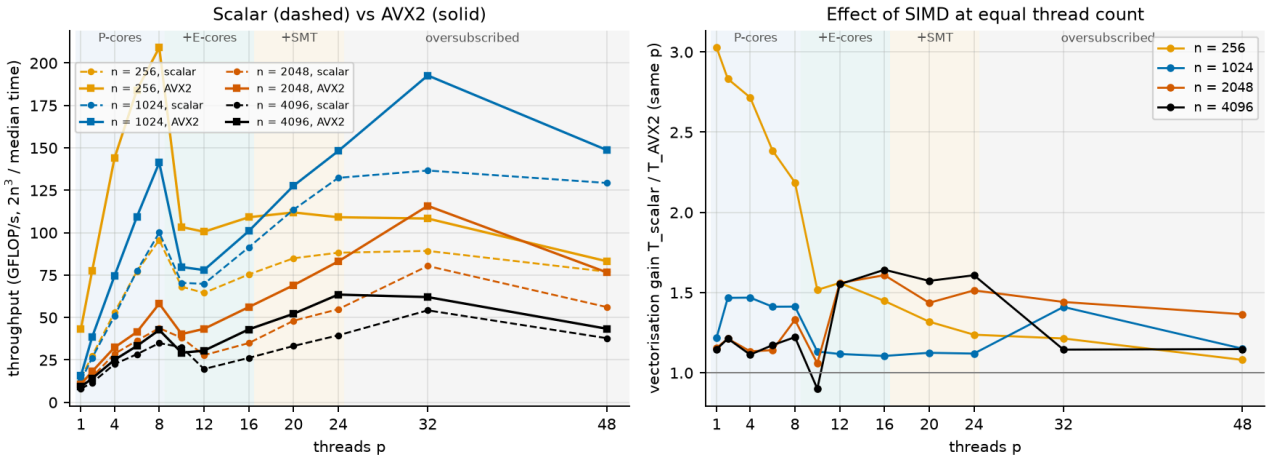


Figure 6. Left: throughput $2n^3/T$ of the scalar (dashed) and AVX2 (solid) builds. Right: vectorisation gain T_{scalar}/T_{AVX2} at the same thread count, schedule and placement.

With one thread, AVX2 is 1.9–3.1× faster for n = 64–512 (3.07× at n = 256), but only 1.22× at 1024 and 1.11–1.21× for n ≥ 1536, far from the factor 4 of the vector width. The thread speed-ups of the AVX2 build (relative to the AVX2 baseline) follow the same pattern as the scalar ones (static drop with E-cores, best at p = 32), reaching 12.2× at n = 1024. At equal p, the SIMD gain at n = 256 falls from 3.0× (p = 1) to 1.24× (p = 24), while for n ≥ 2048 it stays between 0.9 and 1.6×. The best combined result is 18.9× over the scalar sequential baseline at n = 512 and 15.0× at n = 1024 (11.2 ms, 193 GFLOP/s).

4.6 Memory and bandwidth

Table 8. Peak working set of the benchmark process (GetProcessMemoryInfo), scalar build, P1.

n	seq	par p=1	par p=8	par p=24	par p=48	3 × 8n²
256	6.2 MiB	6.3 MiB	6.8 MiB	7.1 MiB	7.6 MiB	1.5 MiB
1024	28.7 MiB	28.8 MiB	29.3 MiB	29.6 MiB	30.1 MiB	24.0 MiB
2048	100.8 MiB	100.8 MiB	101.3 MiB	101.6 MiB	102.1 MiB	96.0 MiB
4096	388.8 MiB	388.9 MiB	389.3 MiB	389.7 MiB	390.2 MiB	384.0 MiB

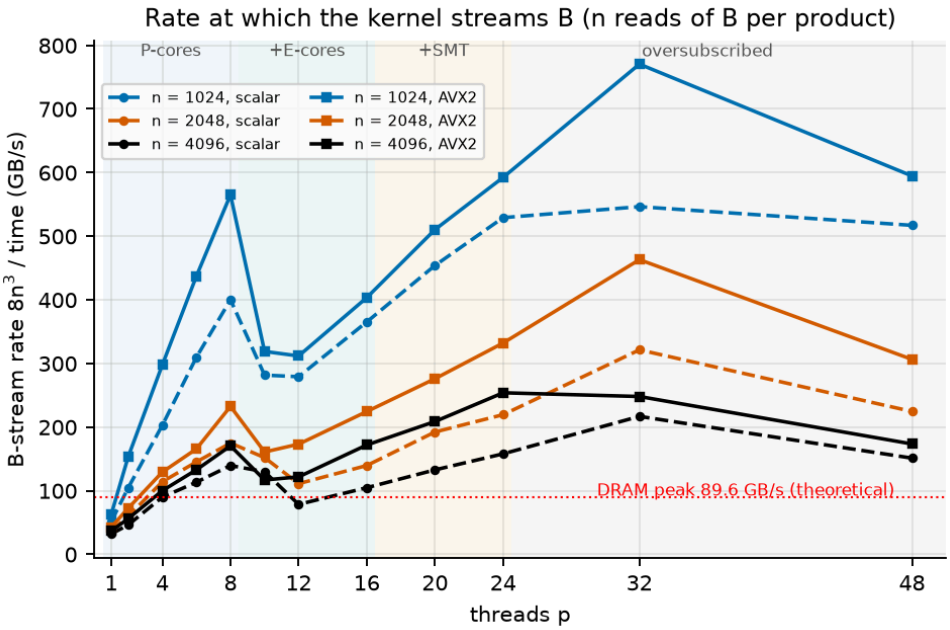


Figure 7. Rate at which the kernel streams B, $8n^3$ bytes (one read of B per row of C) divided by the median time. Values above the theoretical DRAM bandwidth (red) are only possible if part of B is served by the caches.

Memory. The peak working set equals the analytic size of A, B and C ($3 \cdot 8n^2$ bytes) plus a constant of about 5 MiB, and it is independent of the thread count: from the sequential run to 48 threads it grows by 1.4 MiB at n = 4096 (about 30 KiB per thread: touched stack pages and runtime structures). The parallel version allocates no per-thread copy of any matrix. **Bandwidth.** The single-thread kernel streams B at 51 GB/s (n = 1024, B = 8 MiB, L3 resident) and 32 GB/s (n = 4096, B = 128 MiB). With 8 P-core threads the aggregate rate is 400 GB/s at n = 1024 but 140 GB/s at n = 4096, already above the 89.6 GB/s DRAM peak.

4.7 Unusual results and supplementary checks

Table 9. Supplementary checks (`scripts/supplementary_checks.py`): median over 3 fresh processes of the per-process median time (ms), and the range across the processes. C1: the sequential kernel confined by the OS to one of four P-cores; C2: n = 128, seq versus par with one thread; C3: AVX2 parallel kernel, n = 1024.

check	build	method	n	p	where	median (ms)	range (ms)
C1	avx2	seq	1024	1	logical 0	137.049	136.267 – 137.137
C1	avx2	seq	1024	1	logical 2	135.408	135.165 – 136.450
C1	avx2	seq	1024	1	logical 4	134.754	134.687 – 134.836
C1	avx2	seq	1024	1	logical 6	133.761	133.600 – 134.397
C1	scalar	seq	1024	1	logical 0	166.487	166.174 – 166.778
C1	scalar	seq	1024	1	logical 2	167.263	167.081 – 167.715
C1	scalar	seq	1024	1	logical 4	168.000	167.228 – 169.071
C1	scalar	seq	1024	1	logical 6	168.244	167.759 – 168.339
C2	avx2	seq	128	1	pcores	0.164	0.106 – 0.200
C2	avx2	par	128	1	pcores	0.116	0.100 – 0.185
C2	scalar	seq	128	1	pcores	0.294	0.294 – 0.295
C2	scalar	par	128	1	pcores	0.303	0.299 – 0.402
C3	avx2	par	1024	1	pcores	137.251	136.007 – 137.587
C3	avx2	par	1024	2	pcores	55.923	55.875 – 56.357
C3	avx2	par	1024	4	pcores	28.802	28.617 – 28.912

Two results needed checking rather than trust: **superlinear speed-up** of the AVX2 build at n = 1024 ($S(2) = 2.43$, $S(4) = 4.72$, $E > 1$; the scalar build shows only 2.03), and **$S(1) = 1.56$** of the AVX2 build at n = 128. (C1) The baseline runs on logical processor 0, which serves more interrupts; confining the sequential process to P-cores 0, 2, 4 or 6 changes its time by at most 2.5 % (AVX2) and 1.1 % (scalar), so the core choice does not explain a 22 % excess. (C3) The superlinear speed-up is reproducible in fresh processes (median $S(2) = 2.45$, $S(4) = 4.77$ relative to p = 1). (C2) At n = 128 the AVX2 times differ between processes by up to 1.9× for *both* seq and par (seq 0.106–0.200 ms), while they are stable inside a process; the scalar sequential runs vary by less than 1 %. The n = 128 AVX2 ratio is therefore a between-process effect (one process per configuration), not an OpenMP effect, and the AVX2 speed-ups at $n \le 128$ are not reliable.

5. Discussion

H1 (P-cores) is supported for $256 \le n \le 1536$ ($E(8) = 0.83$–0.97) and refuted for $n \ge 2048$ ($E(8) = 0.53$–0.59), which is **H4**. The drop coincides with B (32 MiB at n = 2048) exceeding the 30 MB L3 cache, as in Assignment 2 where blocking started to help at the same point. The bandwidth view supports a memory explanation: the sequential kernel already streams B at 32 GB/s at n = 4096, about a third of the DRAM peak, so eight threads that each read B independently would need several times the available bandwidth. The measured aggregate rate is above the DRAM peak, which is only possible if threads reuse lines of B that another thread has just brought into the shared L3 (with the static schedule all threads traverse B in the same k order, starting at the same time). The kernel is therefore limited by the shared L3 and DRAM, not by arithmetic; tiling B for the caches (Assignment 2) would be the remedy. We did not measure cache or memory counters, so this is an interpretation consistent with the timings and the bound, not a direct observation.

H2 (small matrices) is supported. The fork–join cost of an OpenMP parallel loop is microseconds and grows with p; at n = 64 the whole product takes 37 µs, so parallel execution barely pays (best 1.18×) and loses from 10 threads; at n = 128 it pays only up to the 8 P-cores. Parallel execution helps when each thread receives at least a few hundred microseconds of work and the data fit in the caches.

H3 (static schedule on a hybrid CPU) is supported, and quantitatively. With the static schedule every thread receives n/p rows and the product ends when the slowest thread ends. If one E-core works at a fraction s of a P-core, the time is that of n/p rows at speed s, so $S(p) \approx p \cdot s$ whenever an E-core is in the team. With s = 0.44 (one E-core alone, measured in P4 at n = 2048) the model predicts 4.4, 5.3 and 7.1 for p = 10, 12 and 16; at n = 1024 we measured 5.47, 5.42 and 7.10 (close for 12 and 16; for p = 10 the model, with s taken at another size, is 20 % low). At n = 2048 the E-cores slow down further when a cluster is shared (s = 0.24 per E-core with 4 per cluster), and the model gives 12 × 0.24 = 2.9 for p = 12, against 3.02 measured. Dynamic scheduling lets the P-cores take more rows and removes the drop (Section 4.3). The unpinned runs (P3) behave like the dynamic ones up to p = 16, which is consistent with the Windows scheduler moving the threads that are left on E-cores to P-cores that have finished, although we did not trace thread migrations. The "best" static result at p = 32 has the same cause: the placement map shows that threads 24–31 are placed on the P-cores, so each P-core hosts three threads and each E-core one; the share of rows per core then approximates the ratio of their speeds. This is an accidental load balance, with a higher run-to-run spread (Section 4), and the dynamic schedule obtains more without oversubscription. **SMT** adds 6–64 % with the dynamic schedule (n = 1024: 10.4 → 11.0; n = 2048: 7.7 → 12.2), more at the sizes where the kernel waits for memory, consistent with the second hardware thread hiding memory stalls of a bandwidth-bound kernel.

H5 (vectorisation) is supported. The gain is 2–3× while A, B and C fit in the private L1/L2 caches ($n \le 256$–512: B is 0.5–2 MiB) and 1.1–1.2× once B must come from L3 or memory: SIMD shortens the arithmetic but not the data movement, so the kernel moves from compute-bound to bandwidth-bound and the roofline [4] caps the benefit. For the same reason the SIMD gain shrinks when threads are added at small n (the shared caches and the parallel overhead become dominant), and the two effects do not multiply: at n = 1024, 1.22× (SIMD) and 10.6× (threads, scalar) combine to 15.0×, not 13.0×. The superlinear AVX2 speed-up at n = 1024 fits this picture: a single AVX2 thread reaches only 63 GB/s from L3, below what two cores obtain together (per-core limits on outstanding cache misses would be one explanation); the cause is not the core position (C1) nor noise (C3), but without hardware counters it remains unresolved.

Memory. Threads add no data structures; the memory cost of parallelism here is negligible, unlike in a distributed setting (Assignment 4) where operands must be copied.

Limitations. (i) One machine, one compiler (gcc 16.1, `-O2`), one run order, Balanced power plan; turbo frequencies fall when more cores are active, which lowers S(p) and was not controlled. (ii) No hardware performance counters: cache, bandwidth and migration explanations are hypotheses consistent with the data. (iii) One process per configuration; check C2 shows that between-process variation can exceed within-process variation for small AVX2 kernels, so the reported IQR understates it there. (iv) 5–7 repetitions for $n \ge 1024$. (v) The speed-up uses the median of T_{seq} and T_p; their uncertainties are shown only as the quartile range of T_p. (vi) The kernel is not cache-blocked, so the scaling limit at large n is partly a property of this kernel; a blocked parallel kernel or a BLAS library would scale differently. (vii) The core-type results are specific to this hybrid processor and to Windows' scheduler.

6. Conclusions

1. Within the tested scope (i7-13700K, Windows 11, gcc 16.1 `-O2`, binary64, kernel-only timing), the OpenMP row-parallel version is correct and bit-identical to its sequential counterpart for 1–48 threads; its overhead with one thread is at most 6 %.
2. Parallel execution helps from $n \approx 128$–256 on; on the 8 P-cores the speed-up is nearly linear while the data fit in the 30 MB L3 (up to 7.8 for n = 1024) and saturates around 4.2–4.8 when they do not.
3. On a hybrid CPU the thread count alone is not the right control variable: with a static schedule, adding E-cores makes the product slower ($S \approx p \cdot s$), while a dynamic schedule turns all 24 logical processors into a

speed-up of 11–12 (n = 1024–2048) and 9.8 (n = 4096).

4. AVX2 vectorisation gives 2–3× for cache-resident matrices and only 1.1–1.2× for larger ones; combined with threads the best result is 19× (n = 512) over the scalar sequential kernel. Memory does not grow with threads.

Unresolved: the exact cause of the superlinear AVX2 speed-up, the frequency and migration behaviour behind each curve (hardware counters were not available), and how a cache-blocked parallel kernel would scale.

AI-use disclosure

In line with the course policy, the AI assistant Claude (Anthropic) was used to plan the experimental design, draft and debug the C code, the benchmark and analysis scripts and the tests, and to draft and revise the English of this report. All numbers in the report were produced by running the code in the repository (tables and figures are generated by `scripts/analyze.py`); no result was generated by the AI tool. The author is responsible for understanding the code, checking the measurements and references, and defending the conclusions.

Reproducibility materials

Folder `Assignment03_AlejandroSantana_Materials` (and the repository https://github.com/aimarsantana/BigData-Assignment03-MatMul): `src/c/matmul3.c`, `build.ps1` (exact flags of both builds), `README.md` with build and run commands, `config/benchmark_config.json` (sizes, thread counts, schedules, placements, repetitions, budgets), the input generator and manifest, raw samples, per-run logs with the thread placement map (`results/`), validation output, vectorisation evidence, environment and topology description, the supplementary checks, and the scripts that produce all tables and figures. `run_all.ps1` rebuilds, validates, measures, analyses and regenerates this report. Reused: the unroll4 kernel of Assignment 2 (sequential baseline), the ijk loop of Assignment 1, and the input matrices of Assignments 1–2 (same seeds, verified hashes).

References

1. OpenMP Architecture Review Board, *OpenMP Application Programming Interface, Version 5.2*, Nov. 2021.
2. R. Chandra, L. Dagum, D. Kohr, D. Maydan, J. McDonald, and R. Menon, *Parallel Programming in OpenMP*. San Francisco, CA, USA: Morgan Kaufmann, 2001.
3. G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” in *Proc. AFIPS Spring Joint Computer Conf.*, 1967, pp. 483–485.
4. S. Williams, A. Waterman, and D. Patterson, “Roofline: An insightful visual performance model for multicore architectures,” *Commun. ACM*, vol. 52, no. 4, pp. 65–76, 2009.
5. J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.
6. U. Drepper, “What every programmer should know about memory,” Red Hat, Inc., Tech. Rep., Nov. 2007.
7. Microsoft, “GetLogicalProcessorInformationEx function,” “PROCESSOR_RELATIONSHIP structure” and “SetThreadAffinityMask function,” Microsoft Learn, Windows API documentation.
8. Intel Corporation, *Intel 64 and IA-32 Architectures Optimization Reference Manual*, Santa Clara, CA, USA.
9. N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA, USA: SIAM, 2002.
10. Microsoft, “PROCESS_MEMORY_COUNTERS structure (psapi.h)” and “GetProcessMemoryInfo function,” Microsoft Learn, Windows API documentation.
11. Free Software Foundation, “Options that control optimization” (`-ftree-vectorize`, `-ffp-contract`) and “Developer options” (`-fopt-info`), in *Using the GNU Compiler Collection (GCC)*.
12. K. Goto and R. A. van de Geijn, “Anatomy of high-performance matrix multiplication,” *ACM Trans. Math. Softw.*, vol. 34, no. 3, Art. 12, 2008.